Final Processor Presentation

Aiden Song, Brayden Bolthouse, Lis Villanueva, Minjae Koo, Phillip Nguyen

Design Decisions - Front End

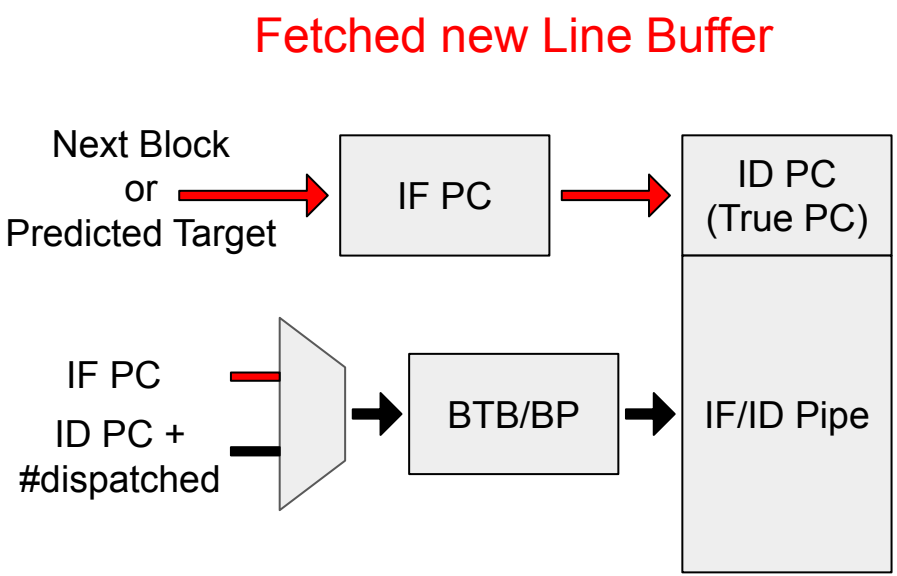
Fetch/PC Handling

True Program Counter is the ID PC

- Dispatches instructions

IF PC runs ahead to the next cache block

- Next block request begins early, a prefetcher of sorts



Instruction Cache/Branch Target Buffer

I-Cache sits between IF and Memory

- Early fetch requests fill the I-Cache, allowing single-cycle access once we are actually ready to fetch
- Intresting quirk - sometimes an early fetch happens to match a load request, causing a single-cycle EXMEM load

2-bit Saturating Counter fills IF/ID with predictions

- Predictions are made from either IF PC or ID PC, ensuring even the cycle after a fetch has predictions already ready

Design Decisions - Back End

Micro-Op Queue - Design Decisions

Split memory ops (addi + mem uOps)

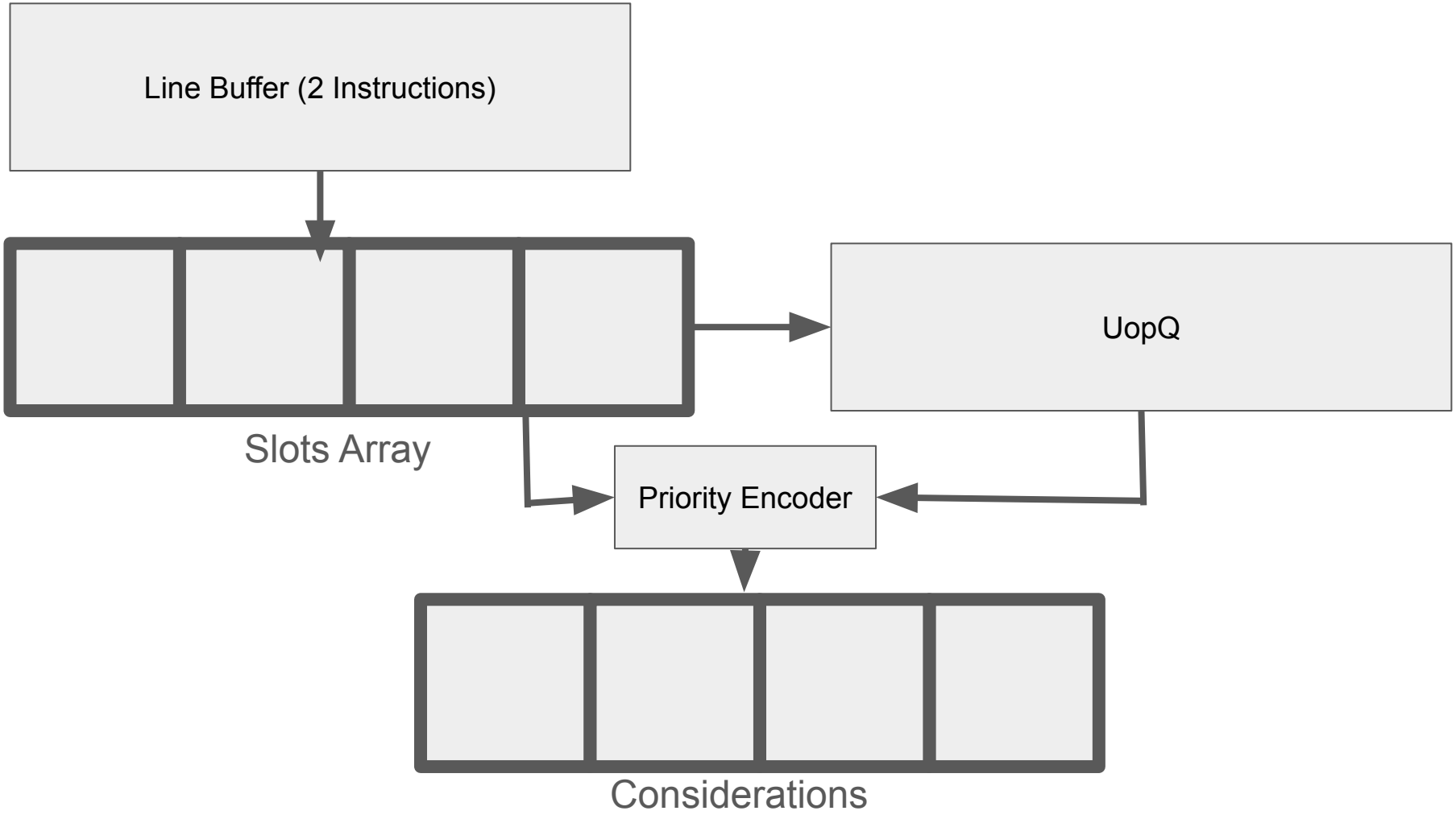
- LW addi rewrites into LW’s Destination
- SW has special registers

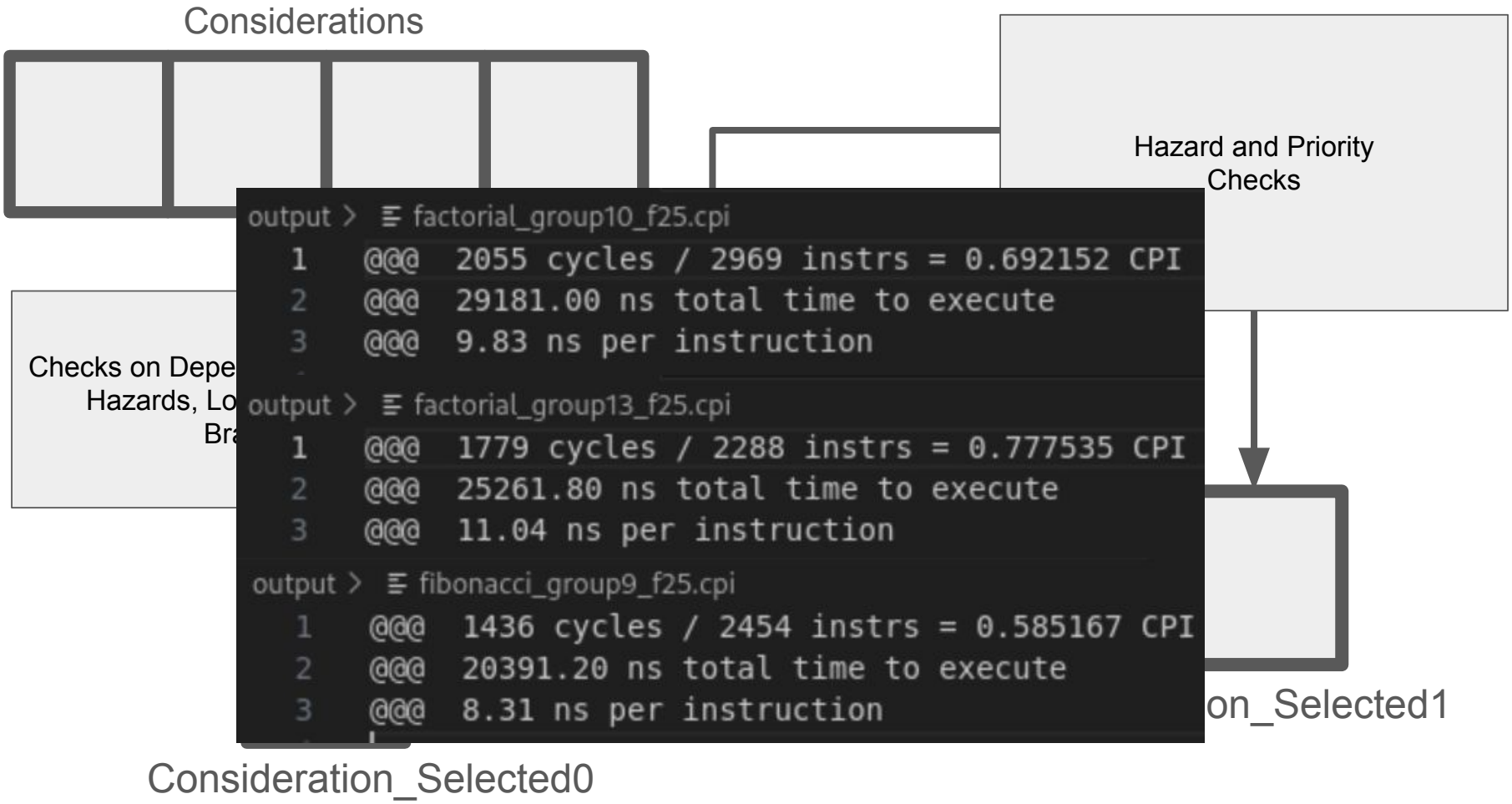
Micro-Op Design

- Slot Array: Instruction -> uOp Translator

Out of Order Implementation

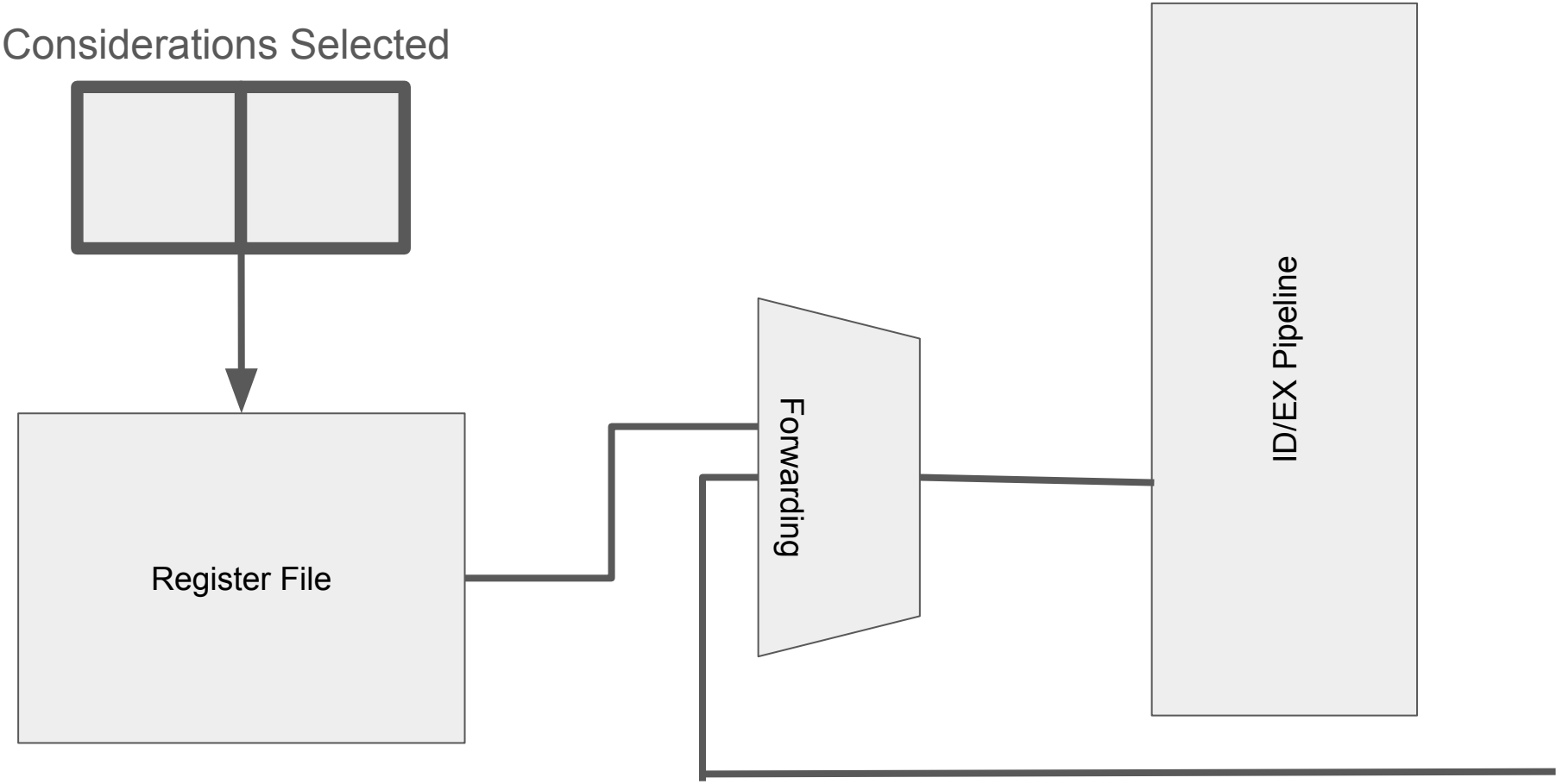
- 4 Wide Lookahead Windows


Line Buffer (2 Instructions)
Slots Array
UopQ
Priority Encoder
Considerations




```
output > factorial_group10_f25.cpi
@@@  2055 cycles / 2969 instrs = 0.692152 CPI
@@@  29181.00 ns total time to execute
@@@  9.83 ns per instruction
output > factorial_group13_f25.cpi
@@@  1779 cycles / 2288 instrs = 0.777535 CPI
@@@  25261.80 ns total time to execute
@@@  11.04 ns per instruction
output > fibonacci_group9_f25.cpi
@@@  1436 cycles / 2454 instrs = 0.585167 CPI
@@@  20391.20 ns total time to execute
@@@  8.31 ns per instruction
```

Considerations Selected



Performance, Area, and Power Metrics - Area-Optimal Sized Structures

Metrics with Small Parameters

Data Structure	Size	Clock Period	CPI (avg of all tests)	TPI (ns)	Area (mm^2)	Power (mW)
I-Cache	4	10.8	1.929554	22.190606	2.121626096	34.7592
BTB	4					
UopQ	8					

Critical path:

exmem_stage/exmem2wb_pipe_reg[destReg][0][1] → exmem_stage/exmem2wb_pipe_reg[alu_result][0][31]

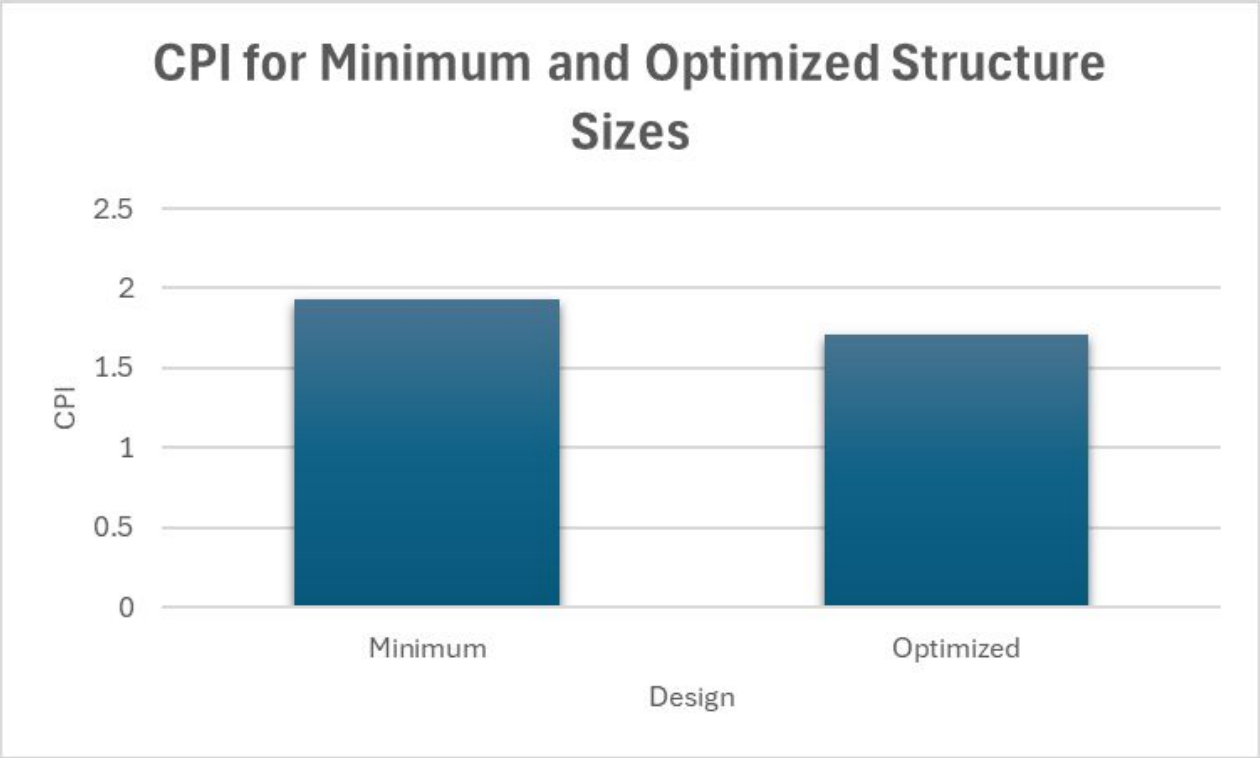
Metrics with Optimized Parameters

Data Structure	Size	Clock Period	CPI (avg of all tests)	TPI (ns)	Area (mm^2)	Power (mW)
I-Cache	16	11.2	1.708639	19.649091	3.949284528	61.1073
BTB	16					
UopQ	8					

Critical path:

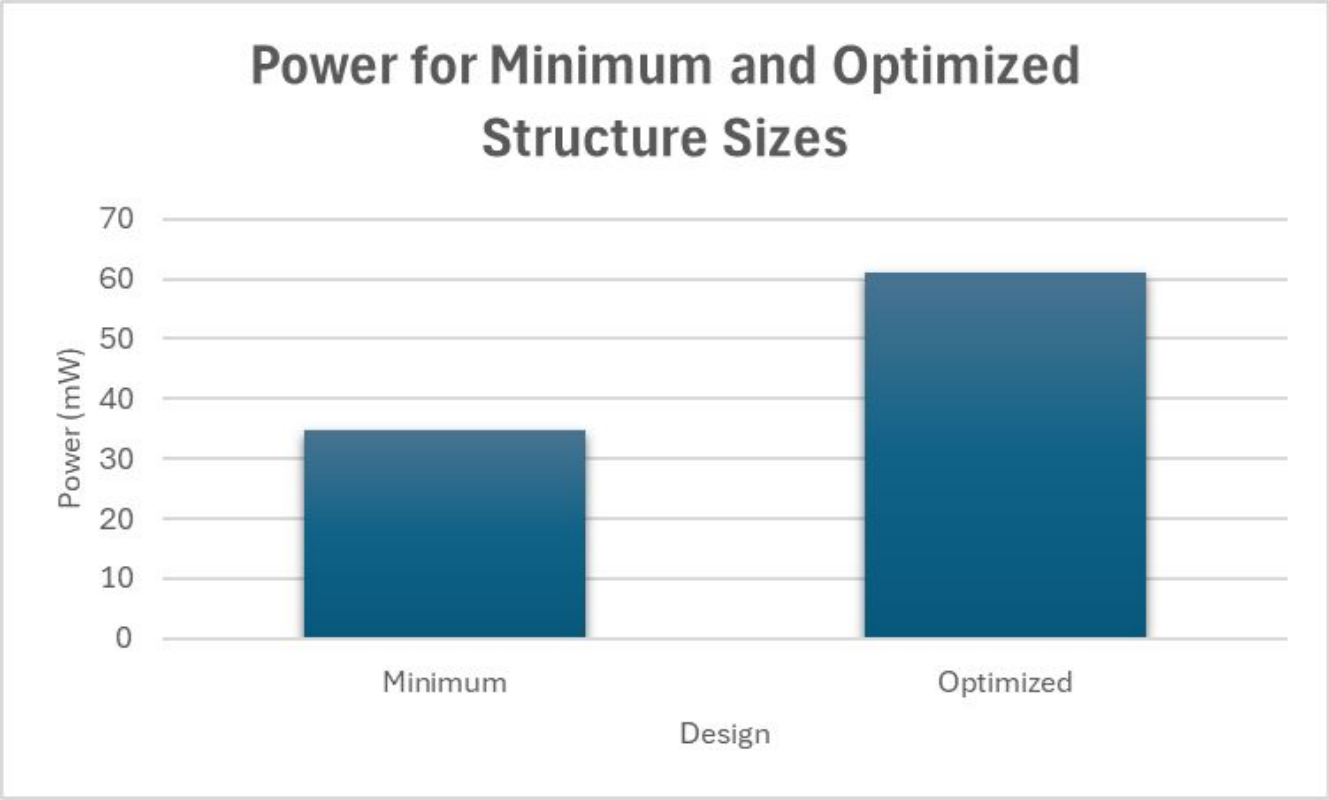
id_stage/id2ex_pipe_reg[regA][0][0] → exmem_stage/exmem2wb_pipe_reg[alu_result][0][31]

CPI

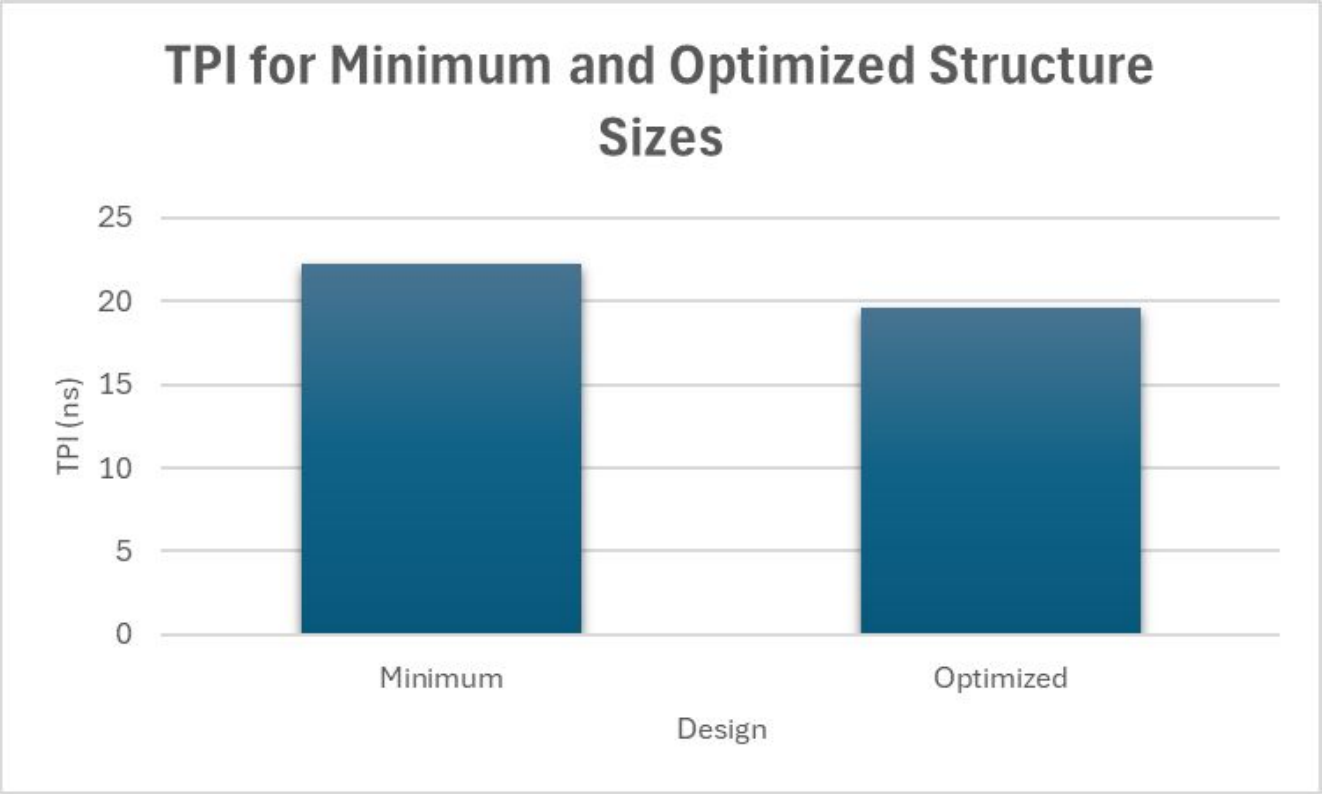


Note: CPI is an average for all programs

Power

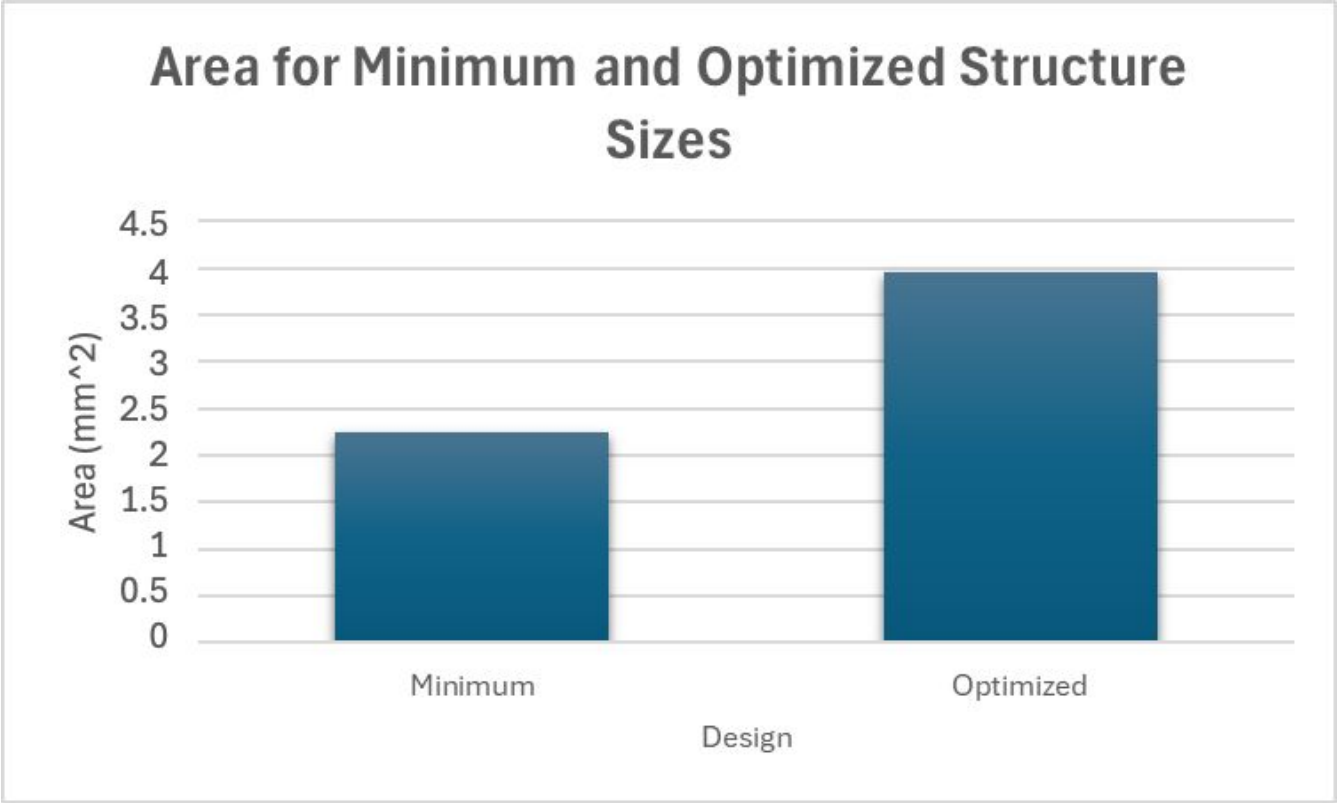

Power for Minimum and Optimized Structure Sizes
70
60
50
40
30
20
10
0
Power (mW)
Minimum
Optimized
Design

TPI



Note: TPI is an average for all programs

Area


Area for Minimum and Optimized Structure Sizes
Area (mm^2)
4.5
4
3.5
3
2.5
2
1.5
1
0.5
0
Minimum
Optimized
Design

Instruction Cache/Branch Target Buffer Sizing

- Swept I-Cache and BTB sizes
- Measured CPI, clock period, area/power

Key Insights

- Larger structures → lower CPI
- But → longer critical path, higher cost

Tradeoff

- Small → fast clock, worse CPI
- Large → better CPI, slower clock

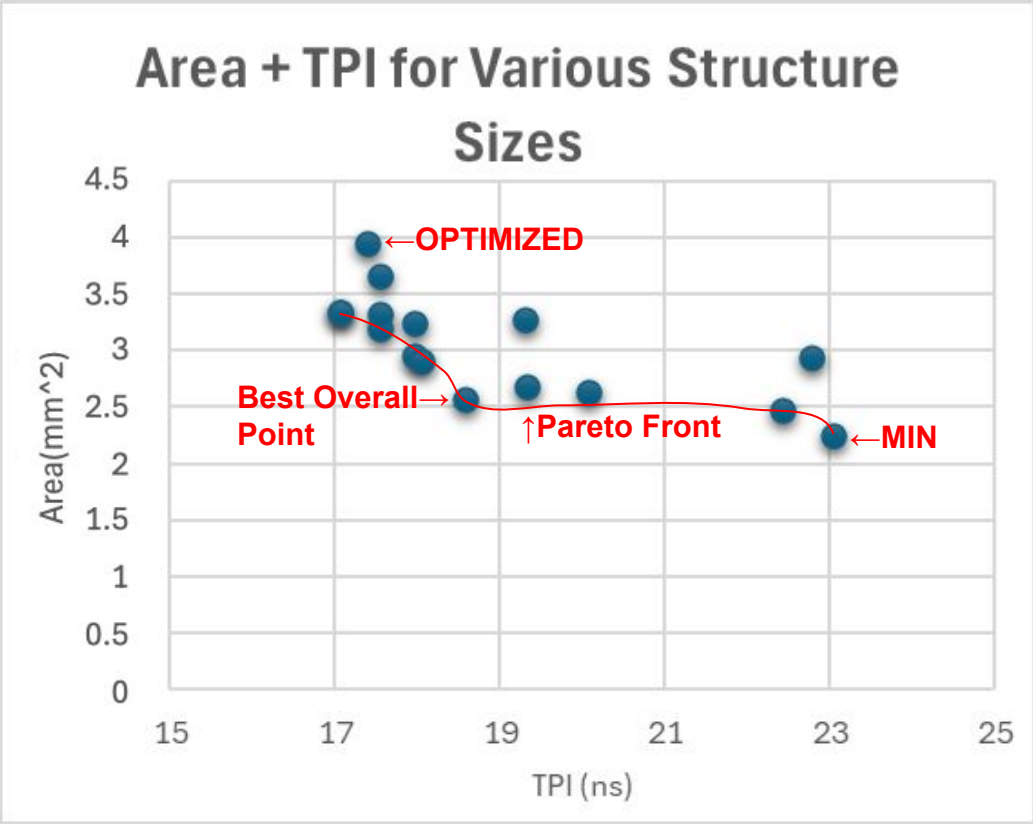
Final choice

- Balanced CPI vs clock → optimize throughput (TPI)

TPI-Area Pareto Chart

Best Point

- **TPI:** 18.57 ns
- **Area:** 2.574 mm^2
- **BTB:** 12 Lines
- **I$:** 4 Blocks



Note: TPI is an average between the 3 programs:
branch_storm_group4_f25, fibonacci_group17_f25, gcd_group15_f25

Instruction Throughput per Area

Different configuration of: I-cache and BTB sizes

Key insight

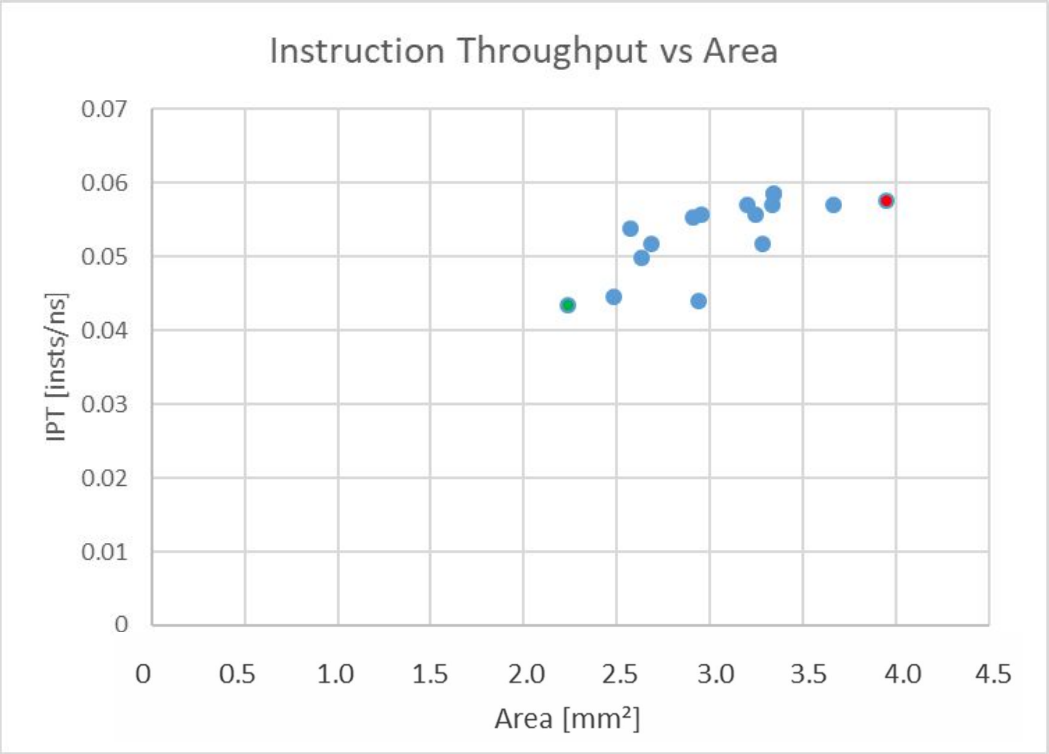
- Large IPT gains up to ~3 mm²
- Diminishing returns beyond ~3.2 mm²

Tradeoff

- Area continues increasing with little performance benefit

Conclusion

- Selected design near the knee of the curve (best efficiency).



Tough Parts

- Edge cases in BTB and UopQ logic
  - Many different hazards in OoO UopQ issue logic
- Instruction count bugs
- Tracing errors through ppln files with many cycles
- Learning synthesizable RTL
- Scheduling with overlapping finals and projects

Lessons Learned

- No replacement for drawing diagrams
- Correctness in OoO execution depends on careful dependency tracking
- Keeping logic cleaner helped debugging
- Decoupling frontend and backend increased complexity but enabled flexible execution

Additional Ideas for EECS 472

Key Additions

- Register renaming → increase physical registers
- ROB → in-order commit, precise state
- Reservation Stations → issue when ready (dynamic scheduling)
- LSQ → safe LW/SW reordering
- Advanced branch prediction → GShare, dynamic, etc

Future Approach - EECS 472

- Plan carefully and deliberately
  - Start with architectural diagrams and always draw out hardware
- A good testbench is very helpful
  - We had a good UopQ testbench which helped us debug easily
- Keep code clean and well-commented
  - Especially in a group setting, documentation is very important
- Always have a working implementation
  - A fast broken processor is useless - golden brick checking matters
- Always show up to work as a team <u>in-person</u>